

Merge Sort

- Merge Sort é um algoritmo de ordenação que divide uma lista em partes menores até que cada parte tenha apenas um elemento.
- Depois, ele junta essas partes novamente em ordem, formando uma lista completamente organizada.

Merge Sort

- Considere o vetor:

[8, 3, 5, 2]

1. Dividir

O vetor é dividido ao meio:

[8, 3] [5, 2]

Depois, cada parte é dividida novamente:

[8] [3] [5] [2]

Merge Sort

2. Combinar (Merge)

Agora os elementos são reunidos em ordem:

$[8] \text{ e } [3] \rightarrow [3, 8]$

$[5] \text{ e } [2] \rightarrow [2, 5]$

Merge Sort

3. Combinar novamente

[3, 8] e [2, 5]

Comparando os elementos em ordem:

2, 3, 5, 8

Resultado:

[2, 3, 5, 8]

Merge Sort

- Imagine que você recebeu um monte de cartas embaralhadas.
- Em vez de tentar organizar todas de uma vez, você:
 1. Divide o monte em montes menores.
 2. Continua dividindo até restar apenas uma carta em cada monte.
 3. Junta os montes comparando as cartas e colocando-as em ordem.
 4. Ao final, todas as cartas estarão organizadas.

Merge Sort

- Utiliza a estratégia "**dividir para conquistar**".
- Sempre divide o vetor ao meio.
- Tem desempenho consistente, mesmo para grandes quantidades de dados.
- Necessita de memória auxiliar para realizar a intercalação (merge).
- Complexidade do Merge Sort:
- $O(n \log n)$
- Essa complexidade ocorre tanto no melhor quanto no pior caso.

Merge Sort x Quick Sort

Quick Sort

Escolhe um pivô

Organiza durante a divisão

Mais rápido na prática

Usa pouca memória extra

Merge Sort

Divide sempre ao meio

Organiza durante a junção

Desempenho mais previsível

Usa memória auxiliar

```
// Função para intercalar (merge) duas partes ordenadas
void merge(int vetor[], int inicio, int meio, int fim) {
    int i, j, k;

    int tamanhoEsq = meio - inicio + 1;
    int tamanhoDir = fim - meio;

    // Vetores auxiliares
    int esquerda[tamanhoEsq];
    int direita[tamanhoDir];

    // Copia os dados para os vetores auxiliares
    for (i = 0; i < tamanhoEsq; i++)
        esquerda[i] = vetor[inicio + i];

    for (j = 0; j < tamanhoDir; j++)
        direita[j] = vetor[meio + 1 + j];

    i = 0; // índice da esquerda
    j = 0; // índice da direita
    k = inicio; // índice do vetor original
```

Merge Sort

Código


```
// Intercala os vetores auxiliares em ordem
while (i < tamanhoEsq && j < tamanhoDir) {
    if (esquerda[i] <= direita[j]) {
        vetor[k] = esquerda[i];
        i++;
    } else {
        vetor[k] = direita[j];
        j++;
    }
    k++;
}
```

```
// Copia elementos restantes da esquerda
while (i < tamanhoEsq) {
    vetor[k] = esquerda[i];
    i++;
    k++;
}
```

Merge Sort

Código

```
// Copia elementos restantes da esquerda
```

```
while (i < tamanhoEsq) {  
    vetor[k] = esquerda[i];  
    i++;  
    k++;  
}
```

```
// Copia elementos restantes da direita
```

```
while (j < tamanhoDir) {  
    vetor[k] = direita[j];  
    j++;  
    k++;  
}
```

Merge Sort

Código

// Função Merge Sort

```
void mergeSort(int vetor[], int inicio, int fim) {  
    if (inicio < fim) {  
        int meio = inicio + (fim - inicio) / 2;  
  
        // Ordena a metade esquerda  
        mergeSort(vetor, inicio, meio);  
  
        // Ordena a metade direita  
        mergeSort(vetor, meio + 1, fim);  
  
        // Junta as duas metades  
        merge(vetor, inicio, meio, fim);  
    }  
}
```

Merge Sort

Código

```

// Imprime o vetor
void imprimirVetor(int vetor[], int tamanho) {
    for (int i = 0; i < tamanho; i++) {
        printf("%d ", vetor[i]);
    }
    printf("\n");
}

int main() {
    int vetor[] = {8, 3, 5, 2};
    int tamanho = sizeof(vetor) / sizeof(vetor[0]);

    printf("Vetor original:\n");
    imprimirVetor(vetor, tamanho);

    mergeSort(vetor, 0, tamanho - 1);

    printf("Vetor ordenado:\n");
    imprimirVetor(vetor, tamanho);

    return 0;
}

```

Merge Sort

Código

Merge Sort Código

- Vetor inicial:

[8, 3, 5, 2]

1. Divisão

O Merge Sort divide o vetor ao meio:

[8, 3] [5, 2]

Divide novamente:

[8] [3] [5] [2]

Merge Sort Código

2. Primeira intercalação (merge)

Junta [8] e [3]:

Comparação:
 $3 < 8$

Resultado:
[3, 8]

Junta [5] e [2]:

Comparação:
 $2 < 5$

Resultado:
[2, 5]

Agora temos:

[3, 8] [2, 5]

Merge Sort Código

3. Intercalação final

Comparações:

3 e 2 $\rightarrow$ 2

3 e 5 $\rightarrow$ 3

8 e 5 $\rightarrow$ 5

resta 8

Resultado:

[2, 3, 5, 8]